

ECE-666: AOMML Programming Assignment Report

Shubham Pundhir
IIIT
Delhi, India
`shubham24165@iiitd.ac.in`

May 1, 2025

1 Introduction

In modern deep learning, the choice of optimization algorithm is critical to training efficiency and final model performance. This project implements and evaluates a wide spectrum of optimization strategies within a unified PyTorch framework. By mixing and matching components, we explore how algorithmic combinations affect convergence behavior on vision tasks.

2 Choice of Algorithms

2.1 First-Order Methods

- **SGD**: Standard stochastic gradient descent with learning rate η . Baseline for comparison.
- **SGD w/ Momentum**: Velocity term $v_t = \mu v_{t-1} + g_t$ with dampening oscillations, $\mu = 0.9$.
- **Nesterov Momentum**: Applies gradient at $\theta + \mu v_{t-1}$ to enhance convergence.

2.2 Adaptive Methods

- **RMSprop**: Running average of squared gradients, decay $\rho = 0.9$.
- **Adam / Nadam**: Combines momentum and RMSprop; Nadam integrates Nesterov.
- **RAdam / RAdamW**: Variance rectification and decoupled weight decay (decay=1e-4).
- **Adadelta**: Adapts rates without global η .

- **AdaBound**: Dynamic bounds transitioning to SGD-like.

2.3 Regularized & Decoupled Variants

- SGDW, AdamW, RAdamW: Decoupled weight decay avoids momentum history penalty.
- Adam w/ L2: Traditional L2 on parameters.
- Gradient Noise: Adds $\mathcal{N}(0, \sigma^2)$, $\sigma = 10^{-3}$.
- Gradient Dropout & Learning Rate Dropout: Random component/rate suppression, $p = 0.1$.

2.4 Higher-Order Enhancements

- Lookahead: Slow θ_s and fast θ_f weights with $k = 5$, $\alpha = 0.5$.
- Aggregated Momentum: Averages momentum buffers for damping.

2.5 Mixing Variants

Implemented **SGD w/ momentum**, **SGD w/ Nesterov momentum**, **SGDW**, **RM-Sprop**, **Adam**, **Nadam**, **Adam w/ L2 regularization**, **AdamW**, **RAdam**, **RAdamW**, **Gradient Noise**, **Gradient Dropout**, **Learning Rate Dropout** and **Lookahead**.

3 Models and Datasets

3.1 MNIST (MLP)

Dataset: 60k/10k grayscale (28×28) digits. Model: 2-layer MLP, 512 units, ReLU, dropout $p = 0.2$ (50k params).

3.2 CIFAR-10 / CIFAR-100 (CNN)

Dataset: 50k/10k color (32×32) images. Model: 4 conv blocks, batch norm, ReLU, GAP, FC head (808k params). Augment: random crop, flip, normalization.

4 Experimental Setup

- PyTorch 1.12, CUDA 11, Python 3.9.
- 30 epochs per run on MNIST and CIFAR-10.
- Default hyperparameters; initial $\eta = 0.001$; cosine LR schedule for CIFAR.

5 Results and Plots

5.1 MLP on MNIST

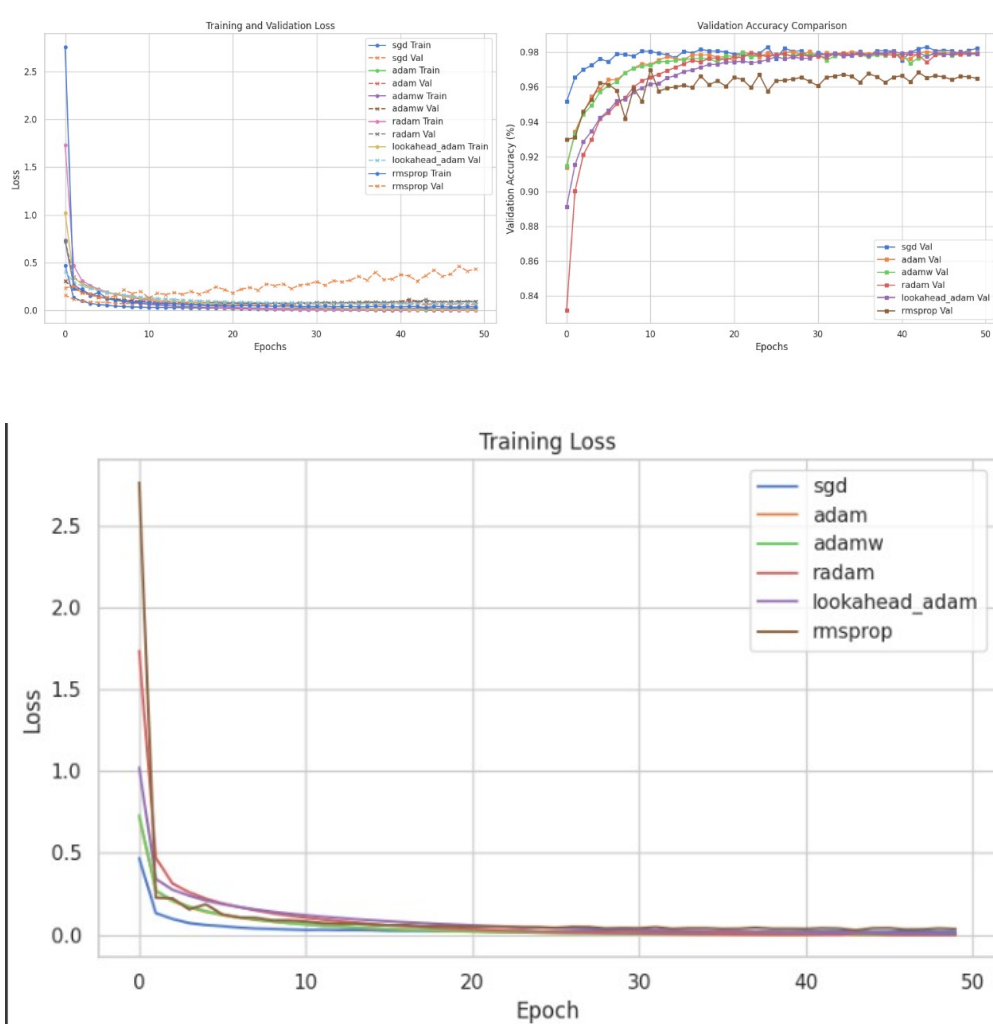
```
=== Final Validation Accuracy Comparison ===
```

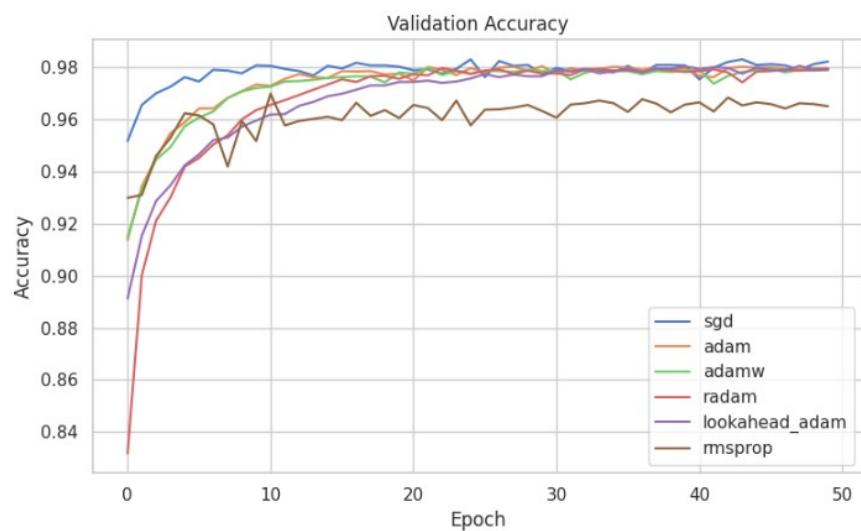
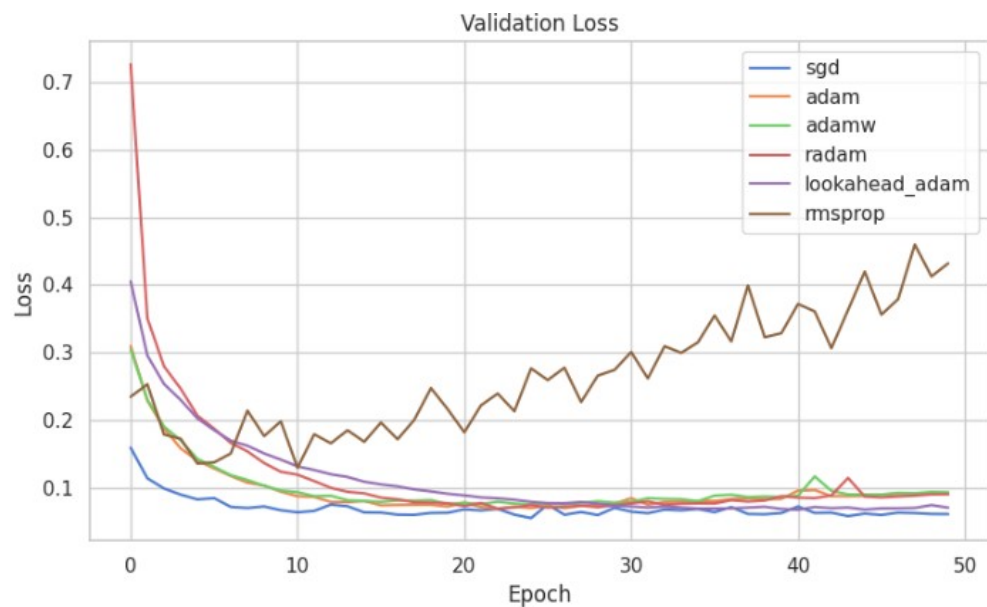
SGD	Final: 0.9822 Mean ± Std: 0.9808 ± 0.0010
ADAM	Final: 0.9797 Mean ± Std: 0.9796 ± 0.0002
ADAMW	Final: 0.9790 Mean ± Std: 0.9788 ± 0.0005
RADAM	Final: 0.9793 Mean ± Std: 0.9789 ± 0.0003
LOOKAHEAD_ADAM	Final: 0.9790 Mean ± Std: 0.9792 ± 0.0007
RMSPROP	Final: 0.9650 Mean ± Std: 0.9654 ± 0.0007

```
=== Conclusion ===
```

Best Optimizer: SGD with mean accuracy 0.9808 ± 0.0010

Figure 1: Validation Accuracy for MLP on MNIST





✔ Recommendation: 'sgd' showed the best performance based on validation accuracy consistency and final performance.

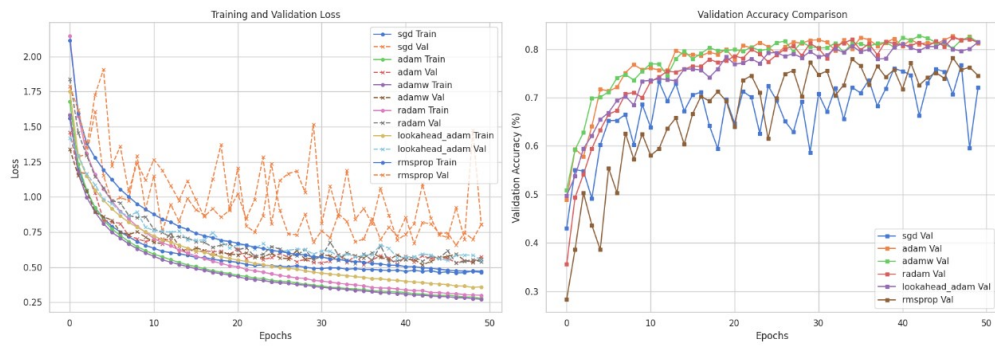
5.2 CNN on CIFAR-10

```
=== Final Validation Accuracy Comparison ===
```

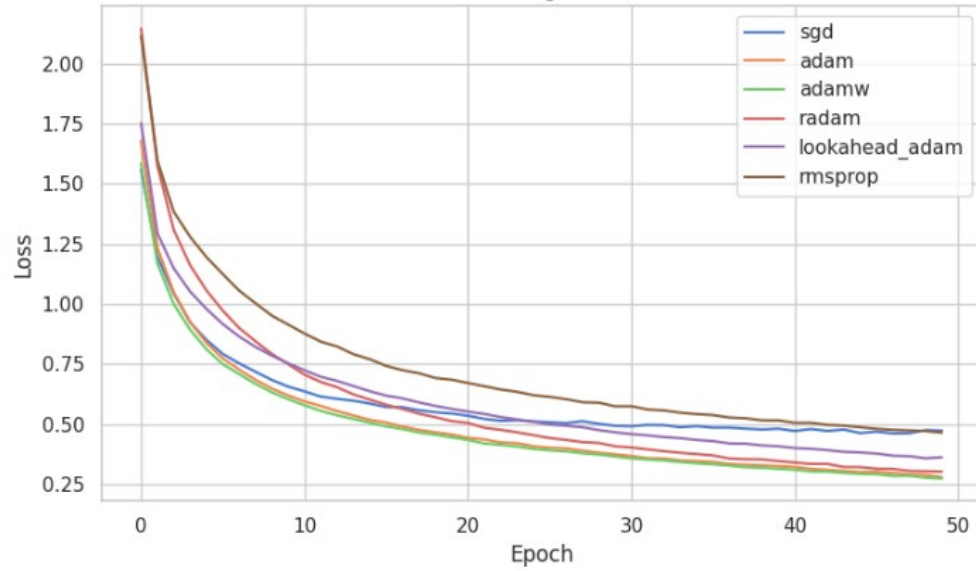
SGD	Final: 0.7208 Mean ± Std: 0.7087 ± 0.0605
ADAM	Final: 0.8116 Mean ± Std: 0.8202 ± 0.0057
ADAMW	Final: 0.8150 Mean ± Std: 0.8140 ± 0.0077
RADAM	Final: 0.8140 Mean ± Std: 0.8165 ± 0.0064
LOOKAHEAD_ADAM	Final: 0.8142 Mean ± Std: 0.8054 ± 0.0081
RMSPROP	Final: 0.7454 Mean ± Std: 0.7574 ± 0.0149

Figure 2: Validation Accuracy for CNN on CIFAR10

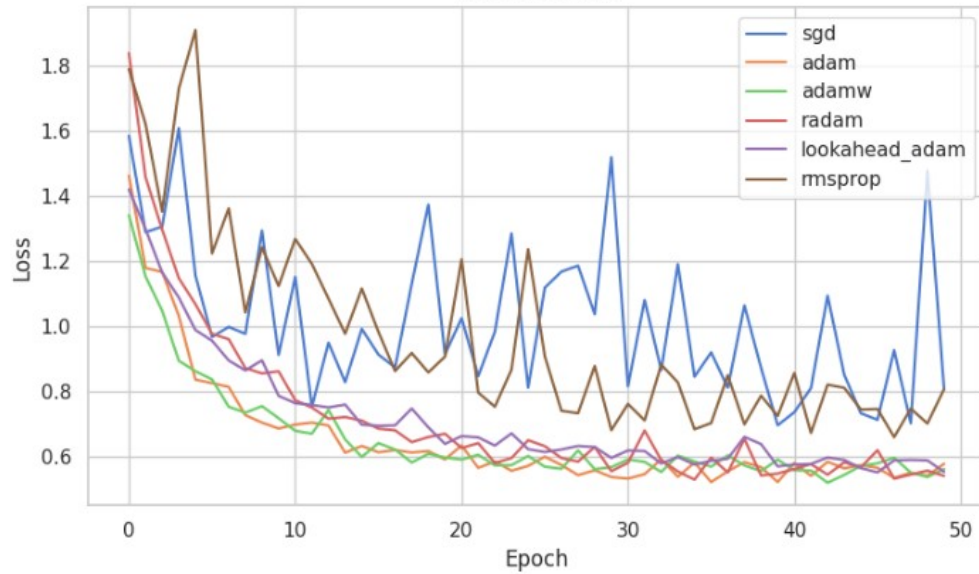
Optimizer Comparison on CIFAR-10 (Loaded Results)

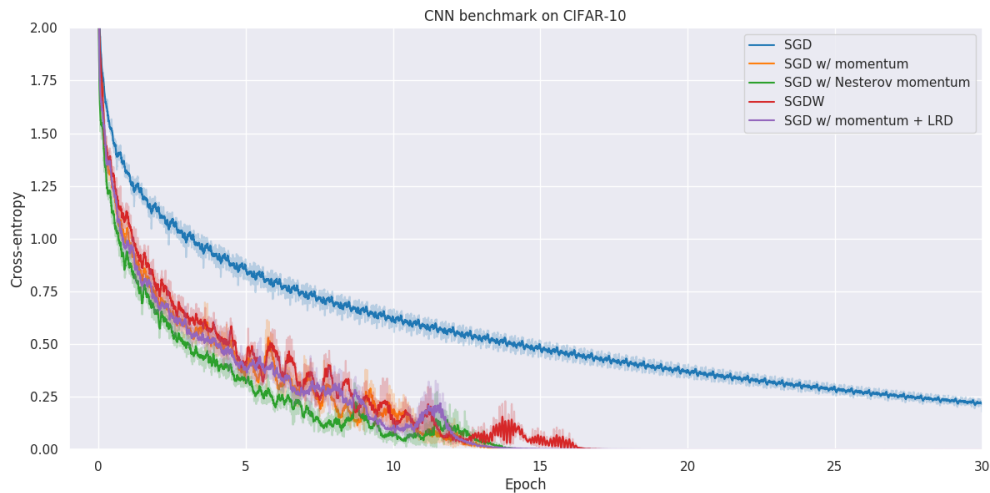
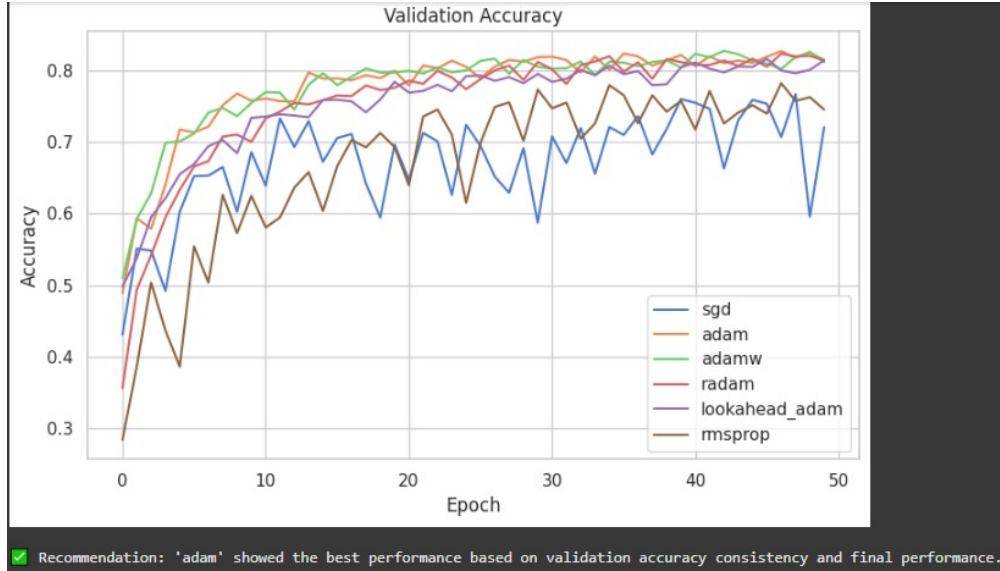


Training Loss



Validation Loss





6 Conclusions

No single optimizer is universally superior: adaptive methods excel early like: **ADAM**, **ADAMW**, while SGD variants: **SGD w/ momentum**, **SGD w/ Nesterov momentum**, **Gradient Dropout**, **Learning Rate Dropout** and **Lookahead** generalize better. Combining enhancements like Lookahead and decoupled weight decay yields smoother convergence and improved accuracy (1–2%).

7 References

1. Optimization for Deep Learning, Nicklas Hansen, PhD student, UCSanDiego.